



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Basi di Dati per la Gestione di un Simulatore di Formula 1

PROGETTO DI BASE DI DATI (2025-2026)
Dipartimento di Matematica

Alessandro Paduret (2150445)

Stefano Andreas Marian Ghetu (2103473)

Indice

1	Abstract	3
2	Analisi dei Requisiti	3
3	Progettazione Concettuale	5
3.1	Lista Entità	5
3.2	Lista Relazioni	6
3.3	Lista Generalizzazioni	7
3.4	Schema E-R	7
4	Progettazione Logica	7
4.1	Analisi delle Ridondanze	7
4.1.1	Con Ridondanza	8
4.1.2	Senza Ridondanza	8
4.1.3	Analisi Risultati	9
4.2	Eliminazione delle Generalizzazioni	9
4.3	Diagramma E-R Ristrutturato	10
4.4	Schema Relazionale	10
4.5	Vincoli referenziali non deducibili dallo schema E-R	11
5	Query	11
5.1	Definizione delle Query	11
5.1.1	Operatori e gestione segnalazioni	11
5.1.2	Incassi per utente	12
5.1.3	Utenti "fit"	12
5.1.4	Posti e prese disponibili	13
5.1.5	Posti liberi	14
5.2	Creazione degli Indici	14
6	Applicazione Software	15
6.1	Struttura generale e moduli	15
6.2	Query prepared vs non prepared	15

1 Abstract

Il progetto definisce un'architettura di database semplificata per la gestione di un servizio di sharing di veicoli urbani (monopattini e biciclette). L'obiettivo è fornire una struttura essenziale e scalabile per tracciare la flotta, gli utenti e i noleggi effettuati in città.

Il sistema modella la gestione della flotta attraverso i veicoli (monopattini e biciclette), le loro caratteristiche specifiche e la collocazione presso hub, punti di ricarica e rastrelliere dislocati nelle diverse zone cittadine.

La parte operativa gestisce gli Utenti, i loro Noleggi e gli spostamenti tra le Zone, registrando le Segnalazioni di malfunzionamento gestite poi dagli Operatori. Ogni noleggio applica una specifica Tariffa, che definisce i costi a cui l'utente è soggetto in base al tipo di veicolo e alla durata dell'utilizzo.

2 Analisi dei Requisiti

Questa base di dati serve a gestire le informazioni relative ad un'azienda che offre un servizio di noleggio urbano, in cui si vogliono sapere i movimenti dei veicoli a disposizione e il lavoro degli operatori dell'azienda che lavorano sul campo.

Veicolo. I veicoli sono tutti i mezzi di sharing che possiede l'azienda. Sono identificati dalla targa e descritti dalle seguenti informazioni:

- Il **Targa** che identifica univocamente il veicolo.
- Lo **Stato** del veicolo (Disponibile, In Uso, Manutenzione).
- La **Marca** del veicolo.

I veicoli sono divisi tra Biciclette e Monopattini.

Bicicletta. Oltre alle informazioni generali dei veicoli, una bicicletta è caratterizzata da:

- Il **Numero Marce** possedute.
- Se è dotata di **Cestino**.

Monopattino. Oltre alle informazioni generali dei veicoli, un monopattino è caratterizzato da:

- Il **Livello di Batteria** che ha il mezzo al momento.
- La **Velocità Massima** a cui il monopattino può andare.
- Il **Peso Massimo** che è in grado di trasportare.

Utente. L'utente rappresenta la Veicolo registrata sull'applicazione di ride sharing. È descritto da:

- La **Email** che la identifica univocamente.
- Il **Nome** e il **Cognome**.
- Il **Metodo di Pagamento** utilizzato.
- Il numero di **Telefono** della Veicolo.
- La sua **Data di nascita**.

Noleggio. L'utente può noleggiare un veicolo alla volta durante un certo lasso di tempo. Un noleggio è descritto da:

- L' **Ora di Inizio** del noleggio.
- L' **Ora di Fine** del noleggio.
- Il **Costo Totale**.

Tariffa. È il costo che viene applicato al minuto durante il noleggio. È identificato unicamente dalla combinazione di **Fascia Oraria** e **Tipo Veicolo**, ed è descritti da:

- La **Fascia Oraria** in cui avviene il noleggio.
- Il **Tipo Veicolo** che si seleziona.
- Il **Costo Sblocco** del veicolo.
- Il **Costo al Minuto**.

Zona. Rappresentano le aree delle varie città in cui il servizio è attivo. La zona è descritta da:

- Il **Nome** della zona.
- La **Città** di riferimento.
- La **Velocità Massima** concessa in quella zona.
- Il **Perimetro** della zona.
- Il **Tipo di Zona** (Attiva, No Parking, Velocità Ridotta).

Segnalazione. I sensori dei veicoli inviano in automatico segnalazioni in caso di malfunzionamenti. È descritto da:

- La **Data e Ora** della segnalazione.
- La **Descrizione** del problema.
- Il **Tipo di Problema**.

Operatore. L'operatore è la Veicolo che lavora presso l'azienda. Sono identificate da un codice fiscale e descritte dalle seguenti informazioni:

- Il **Codice Fiscale** che identifica univocamente la Veicolo.
- Il **Nome e Cognome** della Veicolo.
- Il **Turno** in cui lavorano.

Hub. Il luogo in cui vengono ricaricati i monopattini e avvengono le riparazioni sui veicoli. È identificato univocamente dalla combinazione di **Nome** e **Città**, ed è caratterizzata da:

- La **Capacità Massima** dei veicoli che può ospitare.
- L'**Indirizzo** in cui si trova l'hub.

Punto Ricarica. Sono le stazioni fisiche all'interno degli hub in cui i monopattini vengono caricati. È caratterizzata da:

- Il **Numero presa** della colonnina, univoco internamente all'hub.
- Lo **Stato** della colonnina.
- La **Data dell'Ultimo Test** della colonnina.

Rastrelliera. Sono i punti dove vengono lasciate le biciclette all'interno delle zone quando non sono in utilizzo. È caratterizzata da:

- Il **Codice** della rastrelliera, univoco internamente alla zona.
- Il **Numero Massimo** di posti che possiede.

3 Progettazione Concettuale

3.1 Lista Entità

Il database è formato dalle seguenti entità:

Entità	Descrizione	Attributi	Identificatore
Veicolo	Rappresenta i veicolo che sono gestiti dalla compagnia di sharing.	Targa, Stato, Marca	Targa
Bicicletta	Entita specializzata che estende Veicolo.	NMarce, HasCestino	/
Monopattino	Entita specializzata che estende Veicolo.	LivelloBatteria, VelocitaMax, PesoMax	/
Utente	Rappresenta l'utente registrato sulla piattaforma che noleggia i veicoli.	Nome, Cognome, Email, Telefono, DataNascita, MetodoPagamento	Email
Noleggio	Rappresenta una singola sessione di noleggio attivata da un utente per un veicolo in un intervallo temporale specifico.	DataOraInizio, DataOraFine, CostoTotale, Valutazione	DataOraInizio, Relazione "Effettua", Relazione "Riguarda"
Zona	Rappresenta un'area urbana con regole specifiche di circolazione e sosta.	Nome, Citta, VelocitaMax, TipoZona, Perimetro	Nome, Citta
Tariffa	Definisce il costo applicabile al noleggio in base al tipo di veicolo e alla fascia oraria.	CostoSblocco, CostoAlMinuto, FasciaOraria, TipoVeicolo	FasciaOraria, TipoVeicolo
Operatore	Rappresenta il Veicolole dell'azienda che lavora sui veicoli.	CF, Nome, Cognome, Turno	CF
Segnalazione	Registra un problema o un'anomalia segnalata su un veicolo.	DataOra, Descrizione, TipoProblema	DataOra, Relazione "Genera"
Hub	Luogo dove i veicoli vengono portati per essere riparati e caricati.	Citta, Nome, Indirizzo, CapacitaMax.	Citta, Nome
Rastrelliera	Luogo dove le biciclette vengono lasciate dagli utenti.	Codice,MaxPosti.	Codice, Relazione "Localizzata"
PuntoRicarica	Luogo dove i monopattini vengono messi a caricare.	NPresa, Stato, DataUltimoTest	NPresa, Relazione "Contiene"

3.2 Lista Relazioni

Il database è formato dalle seguenti relazioni:

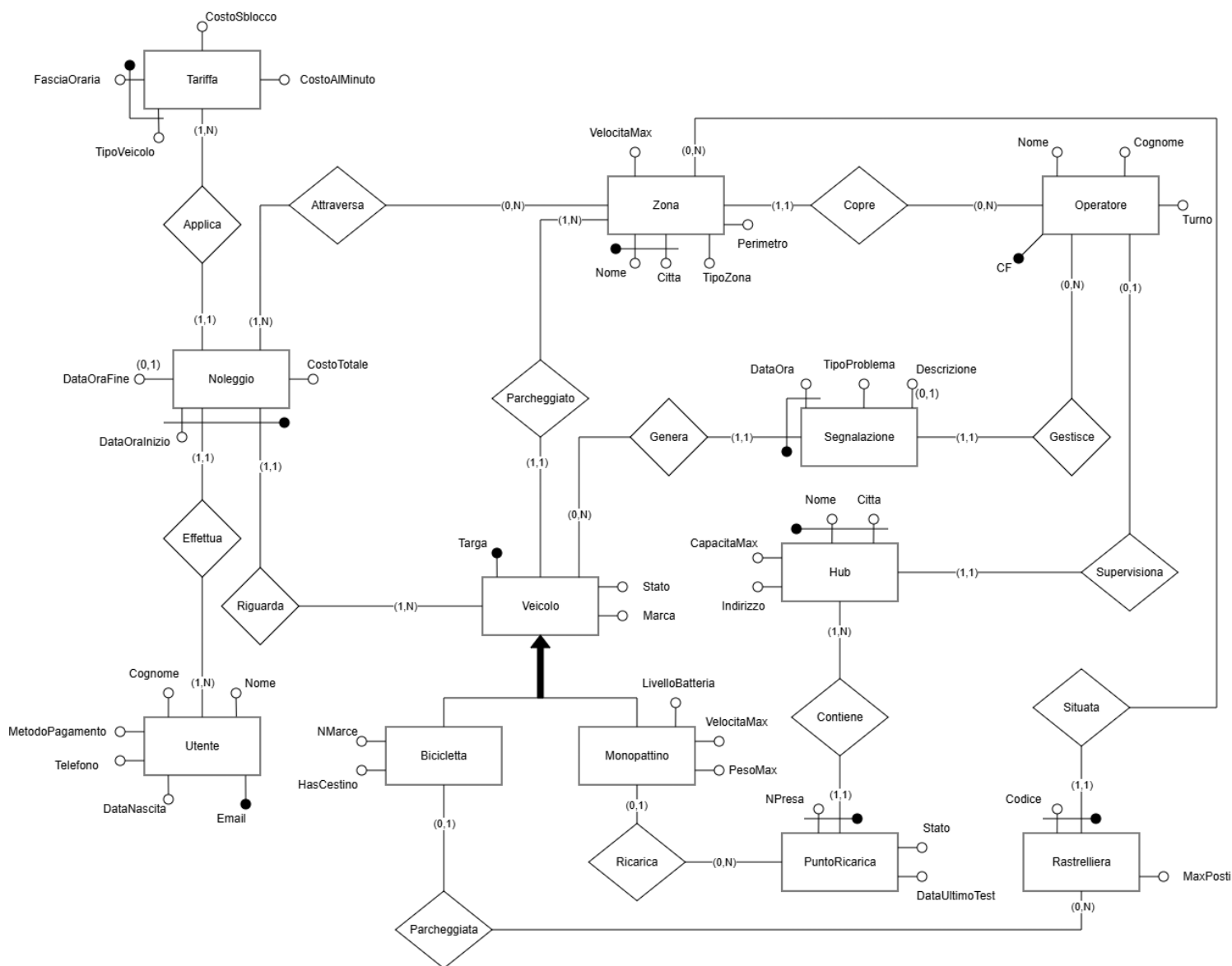
Relazione	Entità Coinvolte	Descrizione	Attributi
Effettua (1:N)	Noleggio(1:1) - Utente(1:N)	– Un utente può effettuare più noleggi nel tempo. – Un noleggio è associato a un solo utente.	/
Riguada(1:N)	Noleggio(1,1) - Veicolo(1,N)	– Un veicolo può essere coinvolto in più noleggi. – Ogni noleggio riguarda un solo veicolo.	/
Applica(1:N)	Noleggio(1,1) - Tariffa(1,N)	– Un noleggio applica una tariffa specifica. – La stessa tariffa può essere applicata a più noleggi.	/
Attraversa(N:N)	Noleggio(1,N) - Zona(0,N)	– Un noleggio può attraversare più zone. – Una zona può essere attraversata da più veicoli noleggiati.	/
Genera(1:N)	Veicolo(0,N) - Segnalazione(1,1)	– Un veicolo può generare più segnalazioni. – Ogni segnalazione riguarda un solo veicolo.	/
Copre(1:N)	Operatore(1,1) - Zona(1,N)	– Un operatore copre una sola zona. – Una zona può essere coperta da uno o più operatori.	/
Gestisce(0:N)	Segnalazione(0,1) - Operatore(0,N)	– Un operatore può gestire più segnalazioni. – Ogni segnalazione viene gestita da un solo operatore.	/
Supervisiona(0:1)	Operatore(0,1) - Hub(1,1)	– Un operatore può supervisionare un hub di ricarica. – Un hub è supervisionato da un operatore.	/
Ricarica(0:N)	Monopattino(0,1) - PuntoRicarica(0,N)	– Un monopattino può necessitare di essere ricaricato. – Un punto di ricarica può ricaricare più monopattini.	/
Contiene(1:N)	PuntoRicarica(1,1) - Hub(1,N)	– Un hub può contenere più punti di ricarica. – Un punto di ricarica appartiene ad un hub.	/
Situata(1:N)	Rastrelliera(1,1) - Zona(0,N)	– Una rastrelliera è situata in una zona. – Una zona può contenere più rastrelliere.	/

Relazione	Entità Coinvolte	Descrizione	Attributi
Parcheggiata(1:N)	Bicicletta(0,1) - Rastrelliera(0,N)	- Una bicicletta può essere parcheggiata. - Una rastrelliera può contenere più biciclette.	/

3.3 Lista Generalizzazioni

- Veicolo è una generalizzazione totale ed esclusiva di Bicicletta e Monopattino.

3.4 Schema E-R



4 Progettazione Logica

4.1 Analisi delle Ridondanze

Analizzando lo schema E-R e la successiva traduzione relazionale, si nota una potenziale ridondanza informativa tra le relazioni Giro e Risultato. La posizione finale presente in Risultato potrebbero infatti essere teoricamente derivati dalla somma dei tempi parziali (Settore1,

Settore2, Settore3) di tutti i giri effettuati da ciascun pilota durante una determinata gara. Riassumendo le operazioni:

- Operazione 1 (4500 volte al giorno): Aggiunta di un nuovo record in Giro.
- Operazione 2 (80 volte al giorno): Visualizzazione delle classifiche delle partite fatte quel giorno.

Durante una gara ci sono 16 piloti per gara e la durata di una gara sarà determinata dal suo numero di giri che in media è di 57.

Quindi per ogni gara dovrò andare ad inserire: $16 \times 57 = 912$ giri. Assumendo che un giocatore va a fare almeno 5 gare al giorno ho circa 4500 inserimenti di giri al giorno.

Poi per i risultati dato che ci sono 16 piloti che giocano una partita, e dal fatto che di media un giocatore fa 5 partite al giorno, ho $16 \times 5 = 80$ risultati che andranno ad essere richiesti da ogni giocatore in totale a fine giornata.

Si assumono i seguenti volumi nella base di dati:

Concetto	Costrutto	Volume
Risultato	R	3200
Gara	E	200
Giro	R	182.400

Numero medio di giri per pilota in una gara stimato: $182.400/3200 = 57$ giri in media.

Si nota anche che dato che **Giro** quando deve essere registrato deve andare a **scrivere** su tutti e tre i settori avrò sempre di base 3 accessi. Si assume infine che il peso delle scritture sia il doppio delle letture.

4.1.1 Con Ridondanza

- **Tabella degli Accessi - Operazione 1**

Concetto	Costrutto	Accesso	Tipo	
Giro	R	3	S	$\times 4500$
Gara	E	1	L	$\times 4500$

Costo Operazione 1: $2 \times (3 \times 4500) + 4500 = \mathbf{31500}$ accessi/giorno.

- **Tabella degli Accessi - Operazione 2**

Concetto	Costrutto	Accesso	Tipo	
Risultato	R	1	L	$\times 80$
Gara	E	1	L	$\times 80$

Costo Operazione 2: $2 \times 80 = \mathbf{160}$ accessi/giorno.

Costo Totale con Ridondanza: $31500 + 160 = \mathbf{31660}$ accessi/giorno.

4.1.2 Senza Ridondanza

- **Tabella degli Accessi - Operazione 1**

Concetto	Costrutto	Accesso	Tipo	
Giro	R	3	S	× 4500
Gara	E	1	L	× 4500

Costo Operazione 1: $2 \times (3 \times 4500) + 4500 = \mathbf{31500}$ accessi/giorno.

- **Tabella degli Accessi - Operazione 2**

Concetto	Costrutto	Accesso	Tipo	
Giro	R	171	L	× 80
Gara	E	1	L	× 80

Costo Operazione 2: $(171 \times 80) + 80 = \mathbf{13.760}$ accessi/giorno.

Qua gli accessi sono 171 perché ci sono i 3 accessi base si Giro × il numero medio di giri che è 57. Andando così a tirare fuori il risultato di un singolo giocatore in una gara.

Costo Totale Strategia B: $31500 + 13.760 = \mathbf{45260}$ accessi/giorno.

4.1.3 Analisi Risultati

Andando quindi a controllare i due casi ho:

- Con ridondanza il costo è: **31660** accessi/giorno.
- Senza ridondanza il costo è: **45260** accessi/giorno.

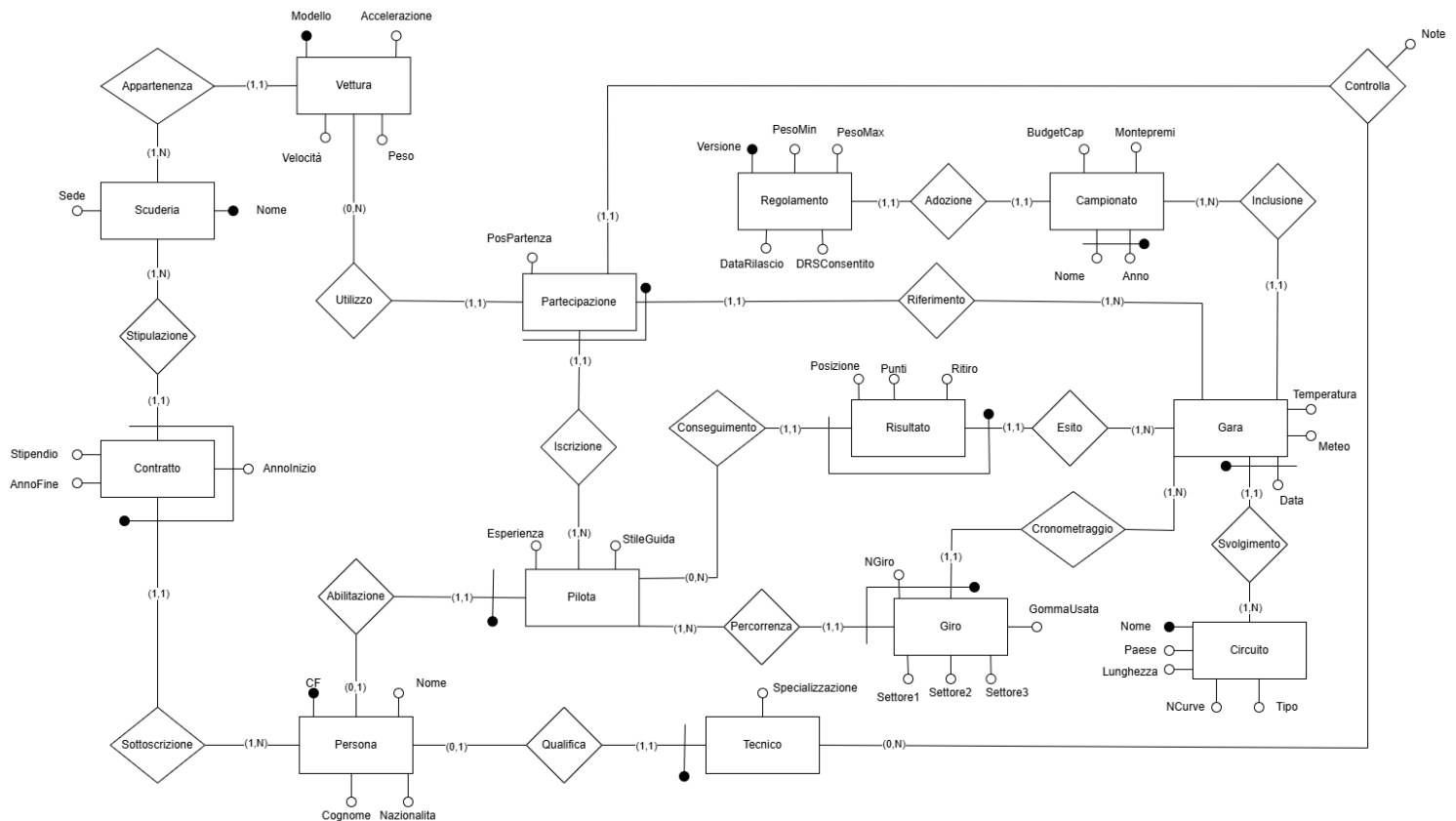
Possiamo notare che il costo senza ridondanza è maggiore rispetto a quella con ridondanza di quasi 15000 accessi, quindi la posizione in risultato deve rimanere per mantenere l'efficienza che l'attributo porta con sé.

4.2 Eliminazione delle Generalizzazioni

Le generalizzazioni presenti nel modello e rappresentate nel diagramma E-R sono state ristrutturare in modo da ridurre al minimo i valori nulli presenti e poter semplificare la successiva implementazione nel database in SQL:

- **Veicolo:** La generalizzazione totale ed esclusiva di Veicolo viene rimossa includendo le relazioni Qualifica e Abilitazione, collegate tramite relazioni 1 : 1. La scelta proviene dal fatto che sia l'entità Bicicletta che l'entità Monopattino sono molto differenziate tra loro tramite i loro rispettivi attributi e hanno delle relazioni particolari che renderebbero il database molto più complesso se entrambe fossero collassate in Veicolo (sarebbe stato necessario controllare che le transazioni fossero effettuate tra Bicicletta e Monopattino). Inoltre, questo metodo evita l'introduzione di ulteriori campi nulli ridondanti, separando le informazioni univoche. Si utilizza la chiave di Veicolo per identificare univocamente anche Bicicletta e Monopattino.

4.3 Diagramma E-R Ristrutturato



4.4 Schema Relazionale

Lo schema logico è rappresentato qui sotto, dove l'asterisco dopo il nome degli attributi indica quelli che ammettono valori nulli.

- **Veicolo** (Targa, Stato, Marca, ZonaCitta, ZonaNome)
 - Veicolo.ZonaCitta → Zona.Citta
 - Veicolo.ZonaNome → Zona.Nome
- **Monopattino** (Veicolo, LivelloBatteria, VelocitaMax, PesoMax, RicaricaHubNome*, RicaricaHubCitta*, RicaricaNPresa*)
 - Monopattino.Veicolo → Veicolo.Targa
 - Monopattino.RicaricaHubNome → PuntoRicarica.HubNome
 - Monopattino.RicaricaHubCitta → PuntoRicarica.HubCitta
 - Monopattino.RicaricaNPresa → PuntoRicarica.NPres
- **Bicicletta** (Veicolo, NMarce, HasCestino, RastrellieraZonaNome*, RastrellieraZonaCitta*, RastrellieraCodice*)
 - Bicletta.Veicolo → Veicolo.Targa
 - Bicletta.RastrellieraZonaNome → Rastrelliera.ZonaNome
 - Bicletta.RastrellieraZonaCitta → Rastrelliera.ZonaCitta
 - Bicletta.RastrellieraCodice → Rastrelliera.Codice
- **Utente** (Email, Nome, Cognome, Telefono, MetodoPagamento, DataNascita)

- Tariffa (FasciaOraria, TipoVeicolo, CostoSblocco, CostoAlMinuto)
- Noleggio (DataOraInizio, Utente, Veicolo, TariffaVeicolo, TariffaOra, DataOraFine*, CostoTotale*)
 - Noleggio.Veicolo → Veicolo.Targa
 - Noleggio.Utente → Utente.Email
 - Noleggio.TariffaVeicolo → Tariffa.TipoVeicolo
 - Noleggio.TariffaOra → Tariffa.FasciaOraria
- Zona (Nome, Citta, TipoZona, Perimetro, VelocitaMax, Operatore)
 - Zona.Operatore → Operatore.CF
- Operatore (CF, Nome, Cognome, Turno)
- Attraversa (NoleggioInizio, NoleggioUtente, NoleggioVeicolo, ZonaCitta, ZonaNome)
 - Attraversa.NoleggioInizio → Noleggio.DataOraInizio
 - Attraversa.NoleggioUtente → Noleggio.Utente
 - Attraversa.NoleggioVeicolo → Noleggio.Veicolo
 - Attraversa.ZonaCitta → Zona.Citta
 - Attraversa.ZonaNome → Zona.Nome
- Segnalazione (DataOra, Veicolo, TipoProblema, Descizione*, Operatore)
 - Segnalazione.Veicolo → Veicolo.Targa
 - Segnalazione.Operatore → Operatore.CF
- Hub (Nome, Citta, CapacitaMax, Indirizzo, Operatore)
 - Hub.Operatore → Operatore.CF
- PuntoRicarica (NPresa, HubNome, HubCitta, Stato, DataUltimoTest)
 - PuntoRicarica.HubNome → Hub.Nome
 - PuntoRicarica.HubCitta → Hub.Citta
- Rastrelliera (Codice, ZonaNome, ZonaCitta, MaxPosti)
 - Rastrelliera.ZonaNome → Zona.Nome
 - Rastrelliera.ZonaCitta → Zona.Citta

4.5 Vincoli referenziali non deducibili dallo schema E-R

- Non si può registrare un giro 5 senza avere tutti i tempi di un giro 3

5 Query

5.1 Definizione delle Query

5.1.1 Operatori e gestione segnalazioni

Obiettivo: Calcolare per ogni operatore il numero di segnalazioni che ha risolto.

```

1 SELECT o.CF, o.Nome, o.Cognome, COUNT(s.Veicolo) AS num_segnalazioni
2 FROM Operatore o
3 LEFT JOIN Segnalazione s ON o.CF = s.CFOperatore
4 GROUP BY o.CF, o.Nome, o.Cognome
5 HAVING COUNT(s.Veicolo) > 0
6 ORDER BY num_segnalazioni DESC

```

Listing 1: Query con GROUP BY, HAVING e operatore aggregato SUM

	cf [PK] character (16)	nome character varying (50)	cognome character varying (50)	num_segnalazioni bigint
1	SANSIL97A82B635X	Silvia	Santoro	4
2	GREDAV73A61B16...	Davide	Greco	3
3	MARNIC76A36B78...	Nicola	Mariani	2
4	FERANT92A72B35...	Antonio	Ferretti	2
5	PALSER76A30B85...	Serena	Palmieri	2
6	GALALE89A46B53...	Alessandro	Gallo	2
7	MARANN97A80B2...	Anna	Marchetti	2
8	ROMSAR71A45B5...	Sara	Romano	2
Total rows: 27		Query complete 00:00:00.075		

5.1.2 Incassi per utente

Obiettivo: Individuare, per ogni utente, gli incassi nell'intervallo temporale derivanti dalle sue prenotazioni.

```

1 SELECT u.*, SUM(n.CostoTotale) AS guadagno
2 FROM Noleggio AS n
3 JOIN Utente AS u ON n.UtenteEmail = u.Email
4 WHERE
5 n.DataOraFine BETWEEN '2024-01-01 00:00:00' AND '2024-12-31 23:59:59'
6 GROUP BY u.email
7 HAVING SUM(n.CostoTotale) > 0
8 ORDER BY guadagno DESC;

```

Listing 2: Query con HAVING, SUM e parametrizzazione in base all'intervallo temporale

	email [PK] character varying (100)	nome character varying (50)	cognome character varying (50)	telefono character varying (20)	datanascita date	metodopagamento character varying (30)	guadagno numeric
1	roberta.ferrara@email.it	Roberta	Ferrara	+393312403258	2003-11-15	paypal	193.89
2	silvia.santoro@email.it	Silvia	Santoro	+393217904996	1996-06-27	carta_credito	193.34
3	elena.marino@email.it	Elena	Marino	+393103488995	1999-03-23	apple_pay	188.99
4	marta.lombardi@email.it	Marta	Lombardi	+393187324253	1986-06-07	carta_debito	188.73
5	alice.giordano@email.it	Alice	Giordano	+393142837990	2002-04-28	carta_credito	185.95
6	eleonora.cattaneo@email.it	Eleonora	Cattaneo	+393233093315	1982-02-11	carta_credito	185.73
7	valentina.conti@email.it	Valentina	Conti	+393491771770	1989-09-04	apple_pay	175.86
8	irene.coppola@email.it	Irene	Coppola	+393257571681	1988-03-15	carta_credito	173.81
Total rows: 50		Query complete 00:00:00.072			LF Ln 8, C		

5.1.3 Utenti "fit"

Obiettivo: Fornire una panoramica degli utenti che vanno in bici e dei loro minuti collezionati in un intervallo di tempo. Note: 'EPOCH FROM (...)' è un convertitore da timestamp a unix time.

```

1 SELECT n.UtenteEmail, ROUND(SUM(EXTRACT(EPOCH FROM (n.DataOraFine - n.
2 DataOraInizio)) / 60)) AS minuti_bici
3 FROM Noleggio n
4 JOIN Bicicletta b ON n.Veicolo = b.Veicolo
5 WHERE
6 n.DataOraFine BETWEEN '2024-01-01 00:00:00' AND '2024-12-31 23:59:59'
7 GROUP BY n.UtenteEmail
8 HAVING ROUND(SUM(EXTRACT(EPOCH FROM (n.DataOraFine - n.DataOraInizio)) /
9 60)) > 0
10 ORDER BY minuti_bici DESC;

```

Listing 3: Query con GROUP BY, HAVING e parametri temporali

	utenteemail character varying (100) 	minuti_bici numeric 
1	silvia.santoro@email.it	460
2	elena.marino@email.it	446
3	nicola.mariani@email.it	421
4	valentina.conti@email.it	413
5	stefano.caruso@email.it	408
6	francesco.fiore@email.it	378
7	alessandro.gallo@email.it	354
8	matteo.colombo@email.it	353
9	eleonora.cattaneo@emai...	341
Total rows: 50		Query complete 00:00:00.0

5.1.4 Posti e prese disponibili

Obiettivo: Fornire una panoramica per ogni città. Si vuole sapere li numero di posti bici e il numero di prese per ricaricare monopattini. Nota: with serve a fare view temporanee che dura 1 query.

```

1 WITH CittaDisponibili AS (
2     SELECT DISTINCT Citta FROM Zona
3     UNION
4     SELECT DISTINCT Citta FROM Hub
5 ),
6 ConteggioRastrelliere AS (
7     SELECT ZonaCitta AS Citta, SUM(MaxPosti) AS num_posti
8     FROM Rastrelliera
9     GROUP BY ZonaCitta
10 ),
11 ConteggioPuntiRicarica AS (
12     SELECT HubCitta AS Citta, COUNT(*) AS num_punti_ricarica
13     FROM PuntoRicarica
14     GROUP BY HubCitta
15 )
16 SELECT
17     c.Citta,
18     COALESCE(r.num_posti, 0) AS num_posti,
19     COALESCE(p.num_punti_ricarica, 0) AS num_punti_ricarica
20 FROM CittaDisponibili c
21 LEFT JOIN ConteggioRastrelliere r ON c.Citta = r.Citta
22 LEFT JOIN ConteggioPuntiRicarica p ON c.Citta = p.Citta

```

Listing 4: Posti e prese disponibili

	citta character varying (50) 	num_posti bigint 	num_punti_ricarica bigint 
1	Milano	151	20
2	Padova	144	15
3	Roma	120	15
Total rows: 3		Query complete 00:00:00.066	

5.1.5 Posti liberi

Obiettivo: Calcolare i posti bici liberi per ogni zona.

```

1 WITH CapacitaZona AS (
2     SELECT
3         ZonaCitta,
4         ZonaNome,
5         SUM(MaxPosti) AS posti_totali
6     FROM Rastrelliera
7     GROUP BY ZonaCitta, ZonaNome
8 ),
9 BiciParcheggiate AS (
10    SELECT
11        RastrellieraCitta AS ZonaCitta,
12        RastrellieraNome AS ZonaNome,
13        COUNT(Veicolo) AS bici_presenti
14    FROM Bicicletta
15    WHERE RastrellieraCitta IS NOT NULL
16    GROUP BY RastrellieraCitta, RastrellieraNome
17 )
18 SELECT
19     c.ZonaCitta AS Citta,
20     c.ZonaNome AS Zona,
21     (c.posti_totali - COALESCE(b.bici_presenti, 0)) AS posti_liberi
22 FROM CapacitaZona c
23 LEFT JOIN BiciParcheggiate b ON c.ZonaCitta = b.ZonaCitta AND c.ZonaNome =
24     b.ZonaNome
25 ORDER BY c.ZonaCitta, c.ZonaNome;
```

Listing 5: Posti liberi

5.2 Creazione degli Indici

Indice scelto

```

1 CREATE INDEX idx_noleggio_data_fine ON Noleggio(DataOraFine);
```

Listing 6: Indice per ottimizzare query numero 3

Motivazione: questo indice serve a velocizzare questa operazione presa dalla query 3.

```

1 n.DataOraFine BETWEEN '2024-01-01 00:00:00' AND '2024-12-31 23:59:59'
```

Listing 7: Operazione da velocizzare

	Città character varying (50) 🔒	Zona character varying (50) 🔒	posti_liberi bigint 🔒
1	Milano	Arcella	17
2	Milano	Camin	11
3	Milano	Centro Storico	19
4	Milano	Guizza	13
5	Milano	Mortise	20
6	Milano	Pontecorvo	13
7	Milano	Prato della Valle	17
8	Milano	San Benedetto	12
9	Milano	Stazione	10
10	Milano	Torre	13
11	Padova	Arcella	8
Total rows: 30		Query complete 00:00:00.070	

6 Applicazione Software

6.1 Struttura generale e moduli

- 1) **src/query.c (entry point e menu)**. Contiene la `main()`: legge la connessione (`DB_CONN` o default), apre il DB, mostra un menu e richiama le query, poi chiude la connessione.
- 2) **include/dbcs.h e src/dbcs.c (accesso al DB + query)**. Incapsula `libpq` nella struct `DBF1` e fornisce: `dbcs_connect()/dbcs_disconnect()`, esecuzione/stampa risultati (`dbf1_exec_and_print()`), supporto a scelte interattive. Qui sono anche implementate le 5 query del menu (una funzione per voce).
- 3) **include/table_render.h e src/table_render.c (render tabellare)**. Converte un `PGresult` in tabella ASCII: calcola larghezze, stampa header/righe e tronca valori troppo lunghi.

6.2 Query prepared vs non prepared

Non prepared: query fisse senza parametri via `PQexec` (tramite `dbcs_exec_and_print()`).
Prepared: query con input utente via `PQexecPrepared` (placeholder `$1,$2,...`), usate per selezione circuito/data e per “giro più veloce”.